

Ref

Operating_System_Concepts_8th_EditionA - Ch8

Operating_Systems_Three_Easy_Pieces - Ch13, 15

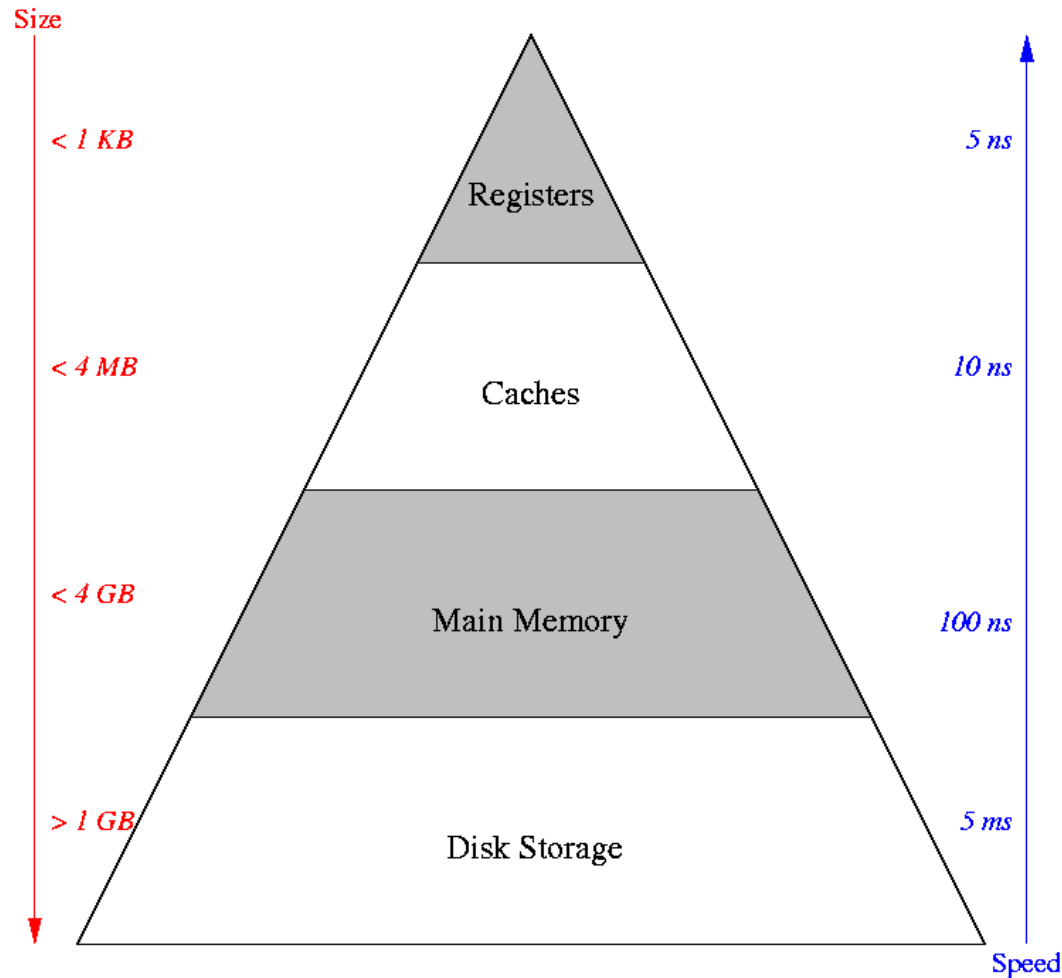
Lecture 4

Memory Concept and Contiguous Memory Allocation

Previously ..

- Scheduling policy
- Basic policy: First Come, First Serve
- Heavyweight resource consumer take first, solution is Shortest Job First
- But arrival time is not the same, solution is Shortest Time-to-Complete First (preemptive scheduler)
- For best response time use Round Robin
- In real-life, we don't know the run-time for each job: use Multilevel Feedback Queue

Review Memory Hierarchy



Introduction

- Can you separate the function of main memory and disk storage?
- The executable program is a binary file in the disk storage that must be brought into the main memory when the programs are executed (their processes)
- The job of the OS is (1): to virtualize memory, to manage and organize the memory
- The jobs of the OS are also (2): to implement virtualized memory in a transparent way for the running program, to make it efficient in terms of time and space, and to protect a process from other processes (memory isolation)

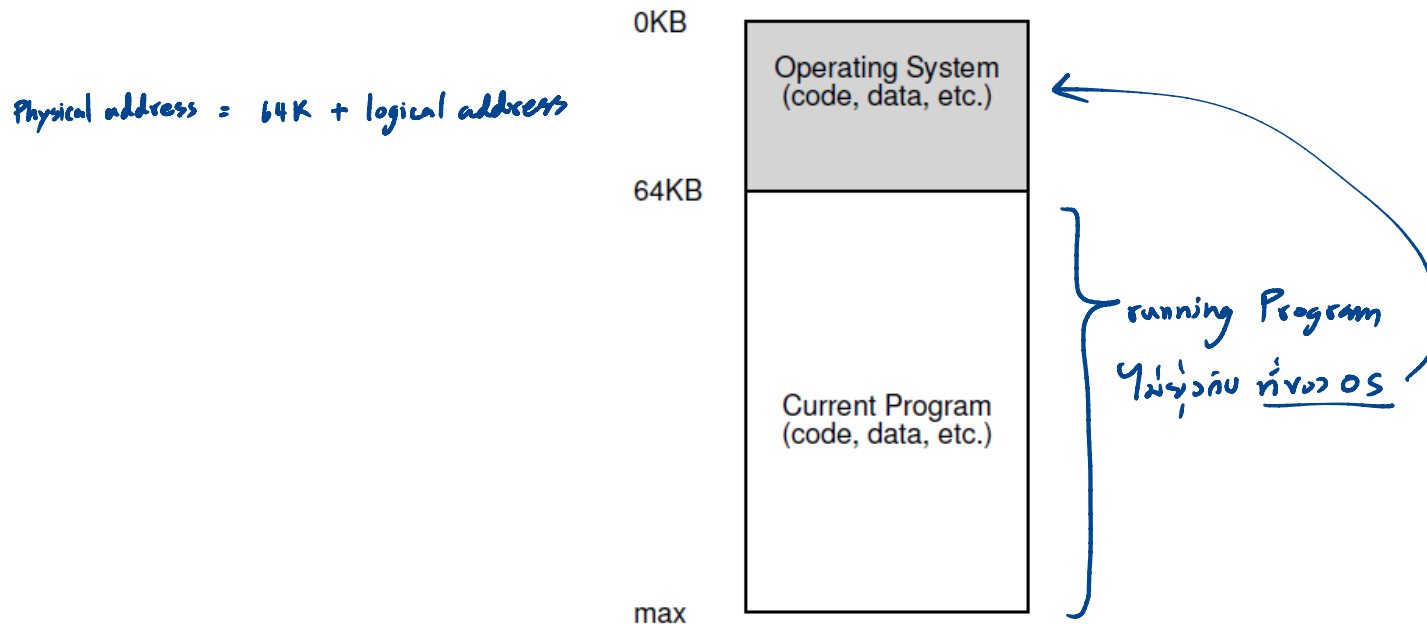
ถ้า process มาทำงาน
ใน memory ของ OS
จะรู้ตัว mem process
หรือไม่

Memory Abstraction

- **Memory Space or Address Space**
 - Programmable Machine (with Main Memory)
 - Time Sharing Machine and Multiprogramming *run multiple programs simultaneously*
- **Physical Address vs. Logical Address**
 - **Physical Address is provided by the H/W** *address by the main mem*
 - A physical address is referred to by the memory unit
 - **Logical Address (Virtual Address) is provided by the OS** *address of process unit*
 - A logical (virtual) address is provided for program instructions and data by the OS
 - The address space is an abstraction of physical memory for processes

Programmable Machine

- One program uses the entire memory space

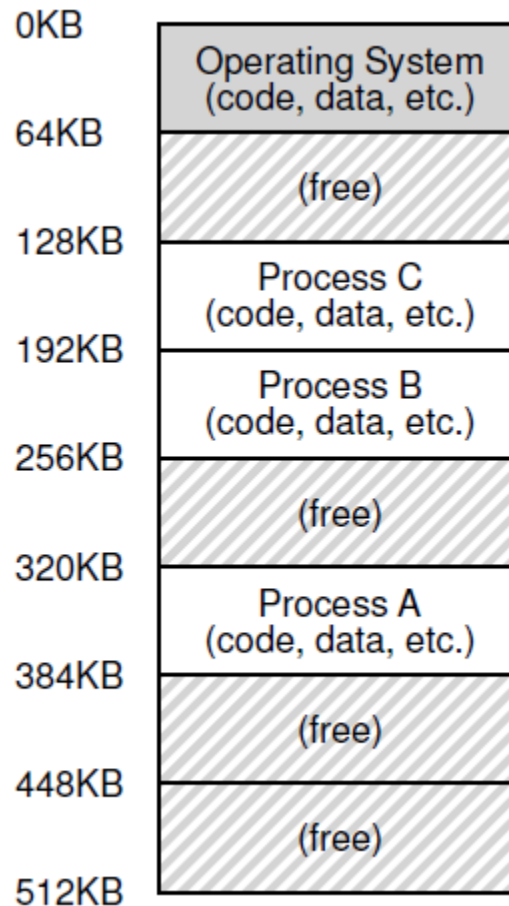


- What is the duty of OS?
 - Mainly a set of libraries of commonly used functions such as I/O handling

Time Sharing Machine and Multiprogramming

process + program
memory

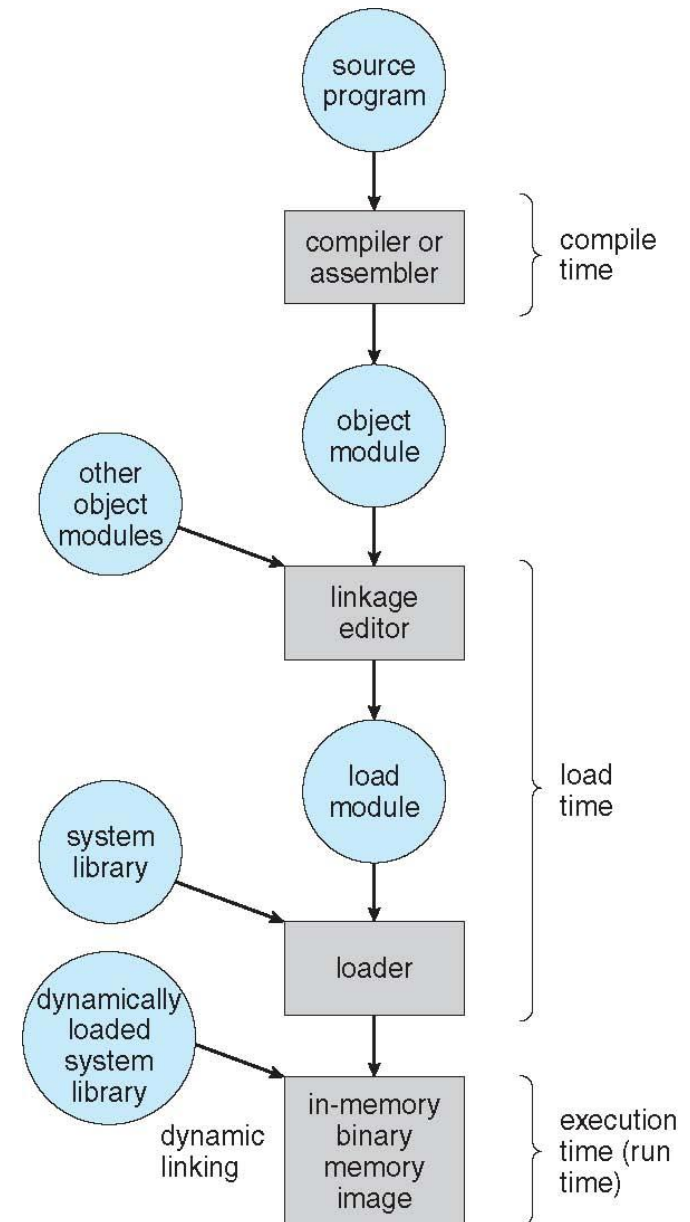
- Each process has its own address space



How address translation?

Address Translation

Translate logical (virtual)
address to physical address
(aka. address binding)



Or Address Binding

- Address binding of instructions and data to physical memory addresses can happen at different stages
 - Compile time: If memory locations known a priori, absolute code can be generated – must recompile code if starting location changes
 - Link/Load time: Can generate relocatable code if memory location is not known at compile time and bind at link/load time
 - Execution time: Binding delayed until run time if the process can be moved during its execution

Compile time
ถ้าผู้เขียนโค้ดรู้ตำแหน่งที่โค้ดจะรันในหน่วยความจำ
หน่วยความจำที่แน่นอนได้ตั้งแต่ตอนคอมไพล์
ถ้าตำแหน่งที่รันโค้ดเปลี่ยนในภายหลัง ต้องคอมไพล์ใหม่
ตัวอย่าง: โปรแกรม MS-DOS รูปแบบ .COM จะถูกคอมไพล์ที่ compile time

Load time
ถ้าไม่รู้ว่าตำแหน่งที่รันโค้ดในหน่วยความจำได้ตั้งแต่ตอนคอมไพล์
การผูกจะเกิดขึ้นตอนโหลดเข้าหน่วยความจำ
ถ้าตำแหน่งที่รันโค้ดเปลี่ยนที่รัน reload โปรแกรมใหม่ ไม่ต้องคอมไพล์ใหม่

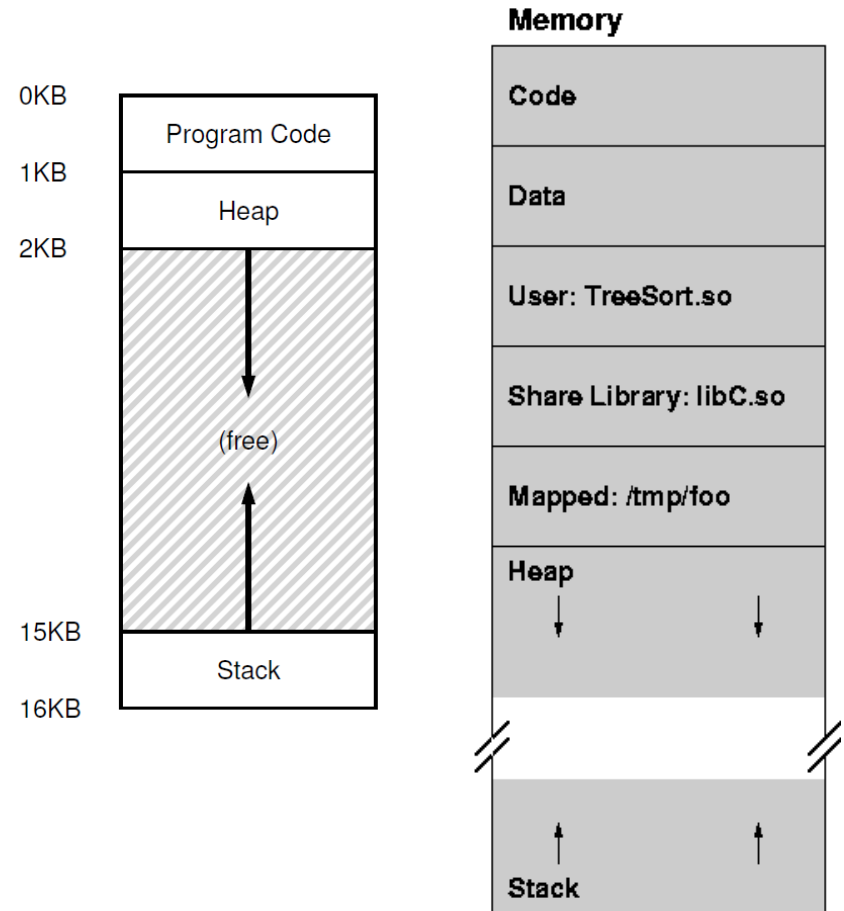
Execution time
ถ้าโปรแกรมสามารถย้ายตำแหน่งในหน่วยความจำระหว่างที่รันอยู่ได้
การผูกจะเกิดขึ้นตอนรัน (runtime binding)
วิธีนี้ต้องอาศัยการคำนวณช่วย
ระบบปฏิบัติการทั่วไปส่วนใหญ่ใช้วิธีนี้

Types of Main Memory

- Programming API Perspective
 - C Storage Classes – Auto, Static, Extern, and Register
 - Defines the scope, life-time, and physical location of variables
 - Memory allocation function *malloc()* used by a programmer to allocate memory dynamically
- OS Perspective
 - Basic memory types are code, stack, and heap segments

Code, Stack, and Heap Segments

- **Code**: used to store program instructions
- **Stack**: used to store data where locations are *implicitly* handled by compiler, e.g. Auto SC
- **Heap**: used to store data where locations are *explicitly* handled by programmer, e.g. allocated using malloc
- Some OSs divide memory into more than 3 segments



Memory Management and Organization

- Why do we need Management and Organization?
- Management Policies
 - Memory Allocation (Process Allocation)
 - Support single or multiple processes
 - Be static or dynamic Allocation
 - **Contiguous** or **Noncontiguous** Allocation
- Organization Issues
 - Memory Placement and Replacement
 - Decides the location in memory where the new incoming data should be placed

Memory Allocation

- **Contiguous Allocation**
 - A process must exist as a single block of contiguous addresses
 - Sometimes it is impossible to find a large enough block
- **Noncontiguous Allocation**
 - A process is divided into chunks called **Pages** or **Segments**
 - Easier to find “holes” in which a page/segment will fit
 - This increases the number of processes that can exist simultaneously in memory (degree of multiprogramming)

CONTIGUOUS MEMORY ALLOCATION

Allocation 2 1100



Single Partition Allocation

support 1 process

Multiple Partition Allocation

Fixed-sized vs. Flexible-sized partitions

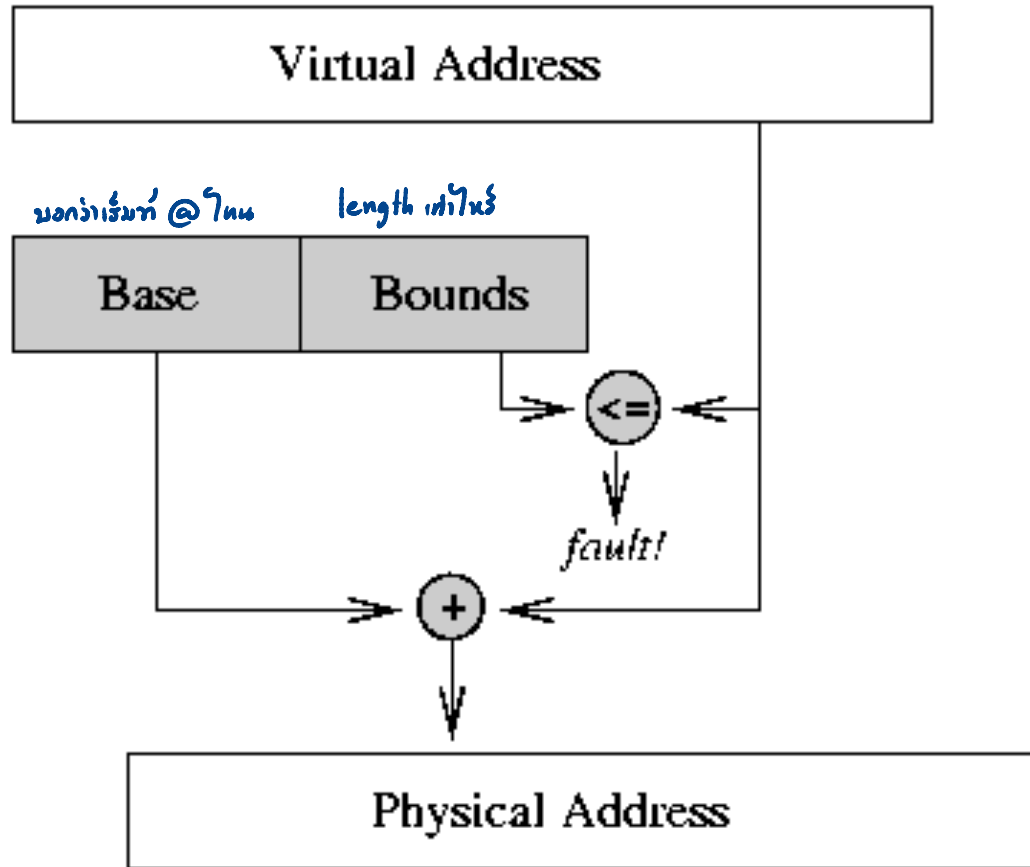
support many process

Single Partition Allocation

- Assume we want to execute a single process
- Assume that the size of the address space is less than the size of physical memory
- Boundary Scheme – Uses a Base and Bounds register pair (also called Relocation and Limit register pair)
 - Contain the smallest address and the range of addresses, respectively
 - Kernel prevents the user process from accessing a virtual address greater than or equal to the Bounds register
- One CPU has one pair of Base and Bounds registers

Boundary Scheme

Base and Bounds



logical @

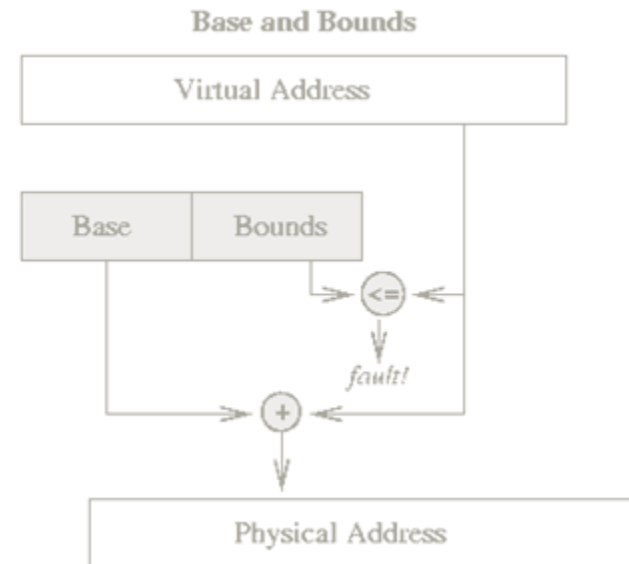
11๗๗๗๗

Physical @

Example

- Suppose the address space of the process is 0 to 4KB-1
- Suppose the Base register contains 16KB and the Bounds register contains 4KB
- The virtual address is compared with the Bounds register contents to check within bounds
- The virtual address is added to the Base register contents to give the physical address

Virtual	→	Physical Address
0	→	16 KB (16384)
1 KB	→	17 KB (17408)
3000	→	19384
4400	→	<i>Fault (out of bounds)</i>



Multiple Partition Allocation

is boundary scheme possible

- Assume we want to execute multiple processes
- Assume again that the size of the address space **is less than** the size of the physical memory
- We allocate one partition (slot) per process
- The degree of multiprogramming is limited by the **number of partitions**
- Address again translated by adding a **relocation value** (~based register!)

Multiple Partition Allocation: Organization Issues

- Fixed-Size: The partitions are all the same size which cannot be changed
 - Static vs. Dynamic Relocation Schemes
- Flexible-Size: All partitions can have a different size

Start with Fixed-sized Partition

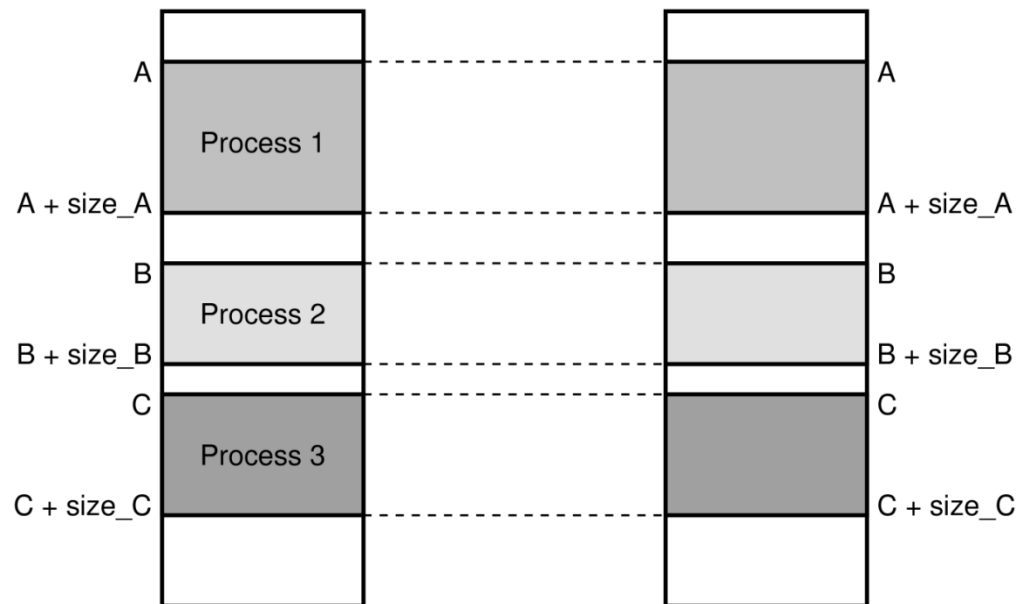
- Can't change the size of partition at run-time
- Still use Base and Bound register pair
- Don't forget: virtual address space is still contiguous in physical memory

Static Relocation

- Static Relocation was an early form of relocation based purely on software methods
- The kernel organizes each process separately where each process has its own relocation value (Base and Bound)
- Using static binding, the loader takes an executable program that is about to be run and rewrites its addresses by adding this relocation value
- This scheme is not very flexible as:
 - There is no protection – processes can generate bad addresses
 - The relocation value cannot be changed easily (need to reload!)

Example of Static Relocation

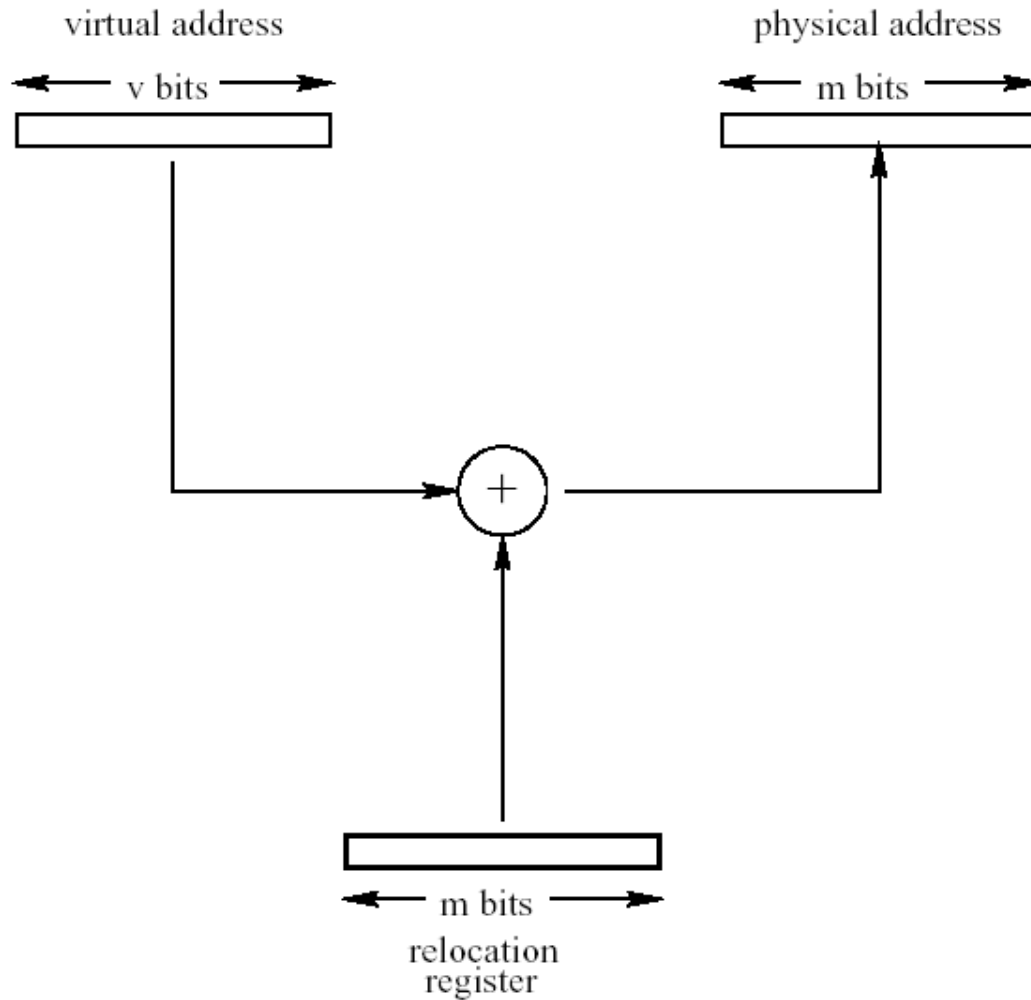
- The instruction move (*movl 1000, %eax*) is performed in which the relocation value is 3000
- Static relocation: the loader rewrites the code by adding the relocation value to the logical address 1000 – new instruction will be *movl 4000, %eax*



Dynamic Relocation

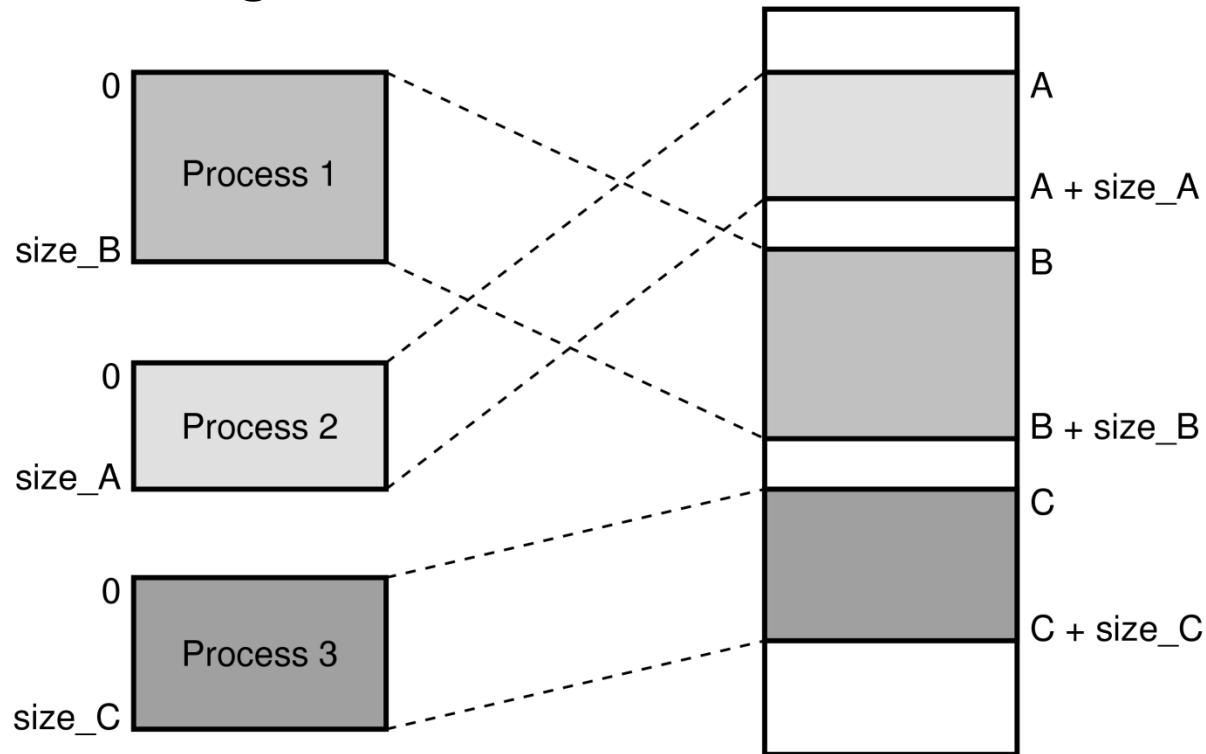
- The kernel organizes each process separately
- Each process has its own relocation value (which can be changed if necessary)
- Using dynamic relocation, the Base and Bounds register pair is **changed when context switching occurs**
 - The hardware should provide **special instructions** to modify the Base and Bounds registers, allowing the OS to change them when different processes run (these instructions can only be executed in kernel mode)

Base (Relocation) Register



Example of Dynamic Relocation

- After context switching, the Base and Bounds register pair should be changed

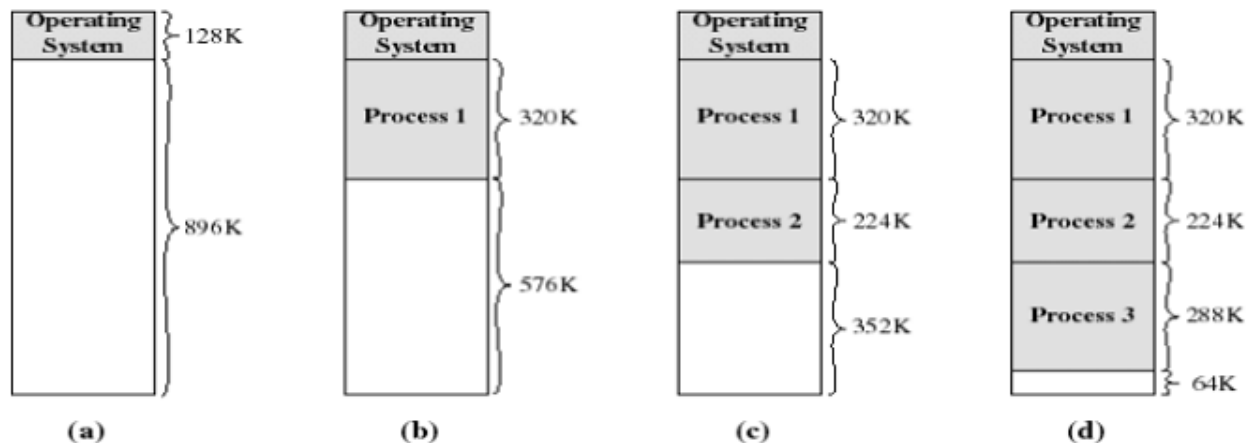


Problem of Fixed-sized Partitions

- An executable program is too small for a partition
 - That mean wasting memory space as some of the partition will be unused – this is called *internal fragmentation* کچھ حصوں کا استعمال نہیں ہوتا
- An executable program is too big to fit a partition

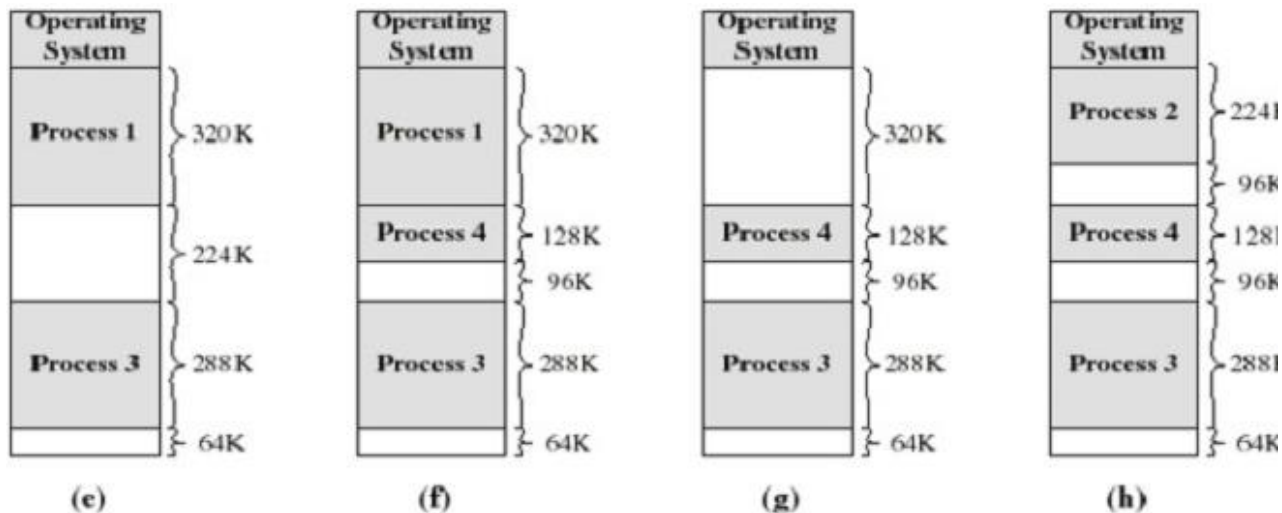
Flexible-size Partitions

- Flexible-sized Partitions
 - Sometimes called variable-sized or dynamic partitions
 - A new process will be allocated immediately after the previous process to eliminate internal fragmentation
 - Suppose the physical memory is 1024KB and the OS takes 128KB
 - Suppose there are a number of processes in the job queue of sizes 320KB, 224KB, 288KB, 128KB, 224KB, 192KB



External Fragmentation

- External Fragmentation occurs when processes are terminated, and no hole spaces are big enough for a new incoming process (suppose we want to execute a process which is 192KB)
- (1)320KB, (2)224KB, (3)288KB, (4)128KB, (5)224KB, (6)192KB



External
fragmentation
พื้นที่ที่ไม่ได้ใช้
(ว่างๆ)

Defragmentation Techniques for External Fragmentation

- Memory Coalescing
 - Combining adjacent free spaces to give a larger single space
- Memory Compaction *107 free space ไม่ต่อเนื่อง*
 - Rearranging free space and occupied space to give a larger single space

Problems of Contiguous Allocation

- Problems with fixed-sized partitions
 - Limited degree of multiprogramming
 - Internal fragmentation
- Problems with flexible-sized partitions
 - External fragmentation (need memory coalescing and compaction)
 - Compaction is possible *only* if relocation is dynamic as the relocation address needs to be changed during execution

ပြင်ဆင်မှု External fragment ဖြစ်ပေါ်မှု Non-contiguous Allocation

Hardware Requirements

- Base/Bounds pair of registers
 - *Need pair per CPU to support address translation and bounds checks*
- Ability to translate virtual addresses and check if within limits
 - *Circuitry to do translations and check bounds limits*
- Privileged instruction(s) to update Base/Bounds registers
 - *Must be able to set these values before letting a program run*
- Ability to raise exceptions
 - *When processes try to access privileged instructions or out-of-bounds memory*
- Privileged instruction(s) to register exception handlers
 - *OS must be able to tell hardware what code to run if exception occurs*

Summary

- When a new process is created, the OS will need to search a data structure (often called a free list) to find room for the new address space of the process and then mark it as used
- When a process terminates, the OS puts its memory back on the free list, and cleans up any associated data structures
- The OS must save and restore the Base and Bounds register pair when it switches between processes
- When a process is stopped (i.e. not running), it is possible for the OS to move an address space from one location in memory to another

Programming API Perspective: C Storage Classes

- A **Storage Class (SC)** defines the scope, life-time, and physical location of a variable and precedes the variable name, e.g. *static int count*
 - *Auto*: the default storage class for all local variables
 - *Static*: used to keep a local variable in existence during the life-time of the program (instead of creating and destroying it each time the function is called and exits)
 - *Extern*: used to give a reference of a global variable that is visible to ALL the program files
 - *Register*: used to define local variables that should be stored in a register instead of RAM

Auto Storage Class

- The Auto SC is used for local variables that are declared within a function or block (block scope)
 - It is the default SC and the *auto* keyword may be omitted
 - The variable is created each time the function is called or the block is entered and it is destroyed on exit

```
printf("Start Program");  
    auto int i=5, j=11;  
    {  
        int k;  
        k=i+j;  
    }  
printf("%d", &k); // k no longer exists
```

*i, j and k are
all auto
variables*